§ 3. Интервальные изменения и отложенные операции

Если прибавить число ко всему массиву, его сумма изменится предсказуемо: к ней добавится произведение числа на длину. Нет необходимости немедленно переписывать каждый элемент. Дерево отрезков позволяет применить это наблюдение к любому диапазону: изменить несколько целиком покрытых узлов и отложить передачу изменений ниже.

Цель главы — не запомнить функцию `push`, а понять, какое состояние она представляет. Мы реализуем присваивание и прибавление на диапазоне, докажем порядок композиции меток и разберём пример с чётными и нечётными суммами из старого конспекта курса.

3.1. Новый контракт требует нового инварианта

Поддерживаем `add(l,r,d)`, `set(l,r,x)` и `sum(l,r)` на полуинтервалах $[l, r)$. Размер массива постоянен; пустой диапазон допустим и ничего не меняет. Значения могут быть отрицательными. Для примеров предполагаем $n \le 10^6$ и представимость всех значений, меток, произведений на длину и промежуточных сумм в `long long`.

После прибавления d к диапазону длины k его сумма возрастает на dk. После присваивания x она становится xk. Значит, для полного покрытия узла ответ можно обновить за постоянное время. Но дети продолжают хранить старые сведения. У узла появляется обязанность: перед будущим спуском передать детям преобразование, которое они ещё не учли.

Обычное равенство «сумма родителя равна сумме хранимых сумм детей» теперь не всегда истинно. Для узла с меткой f верно другое: его сумма соответствует результату применения f к объединению состояний детей. Метка уже учтена в собственной сумме узла, но ещё не учтена в его детях. Вдоль пути могут существовать несколько таких отложенных преобразований; доступ к поддереву должен учитывать их в правильном порядке.

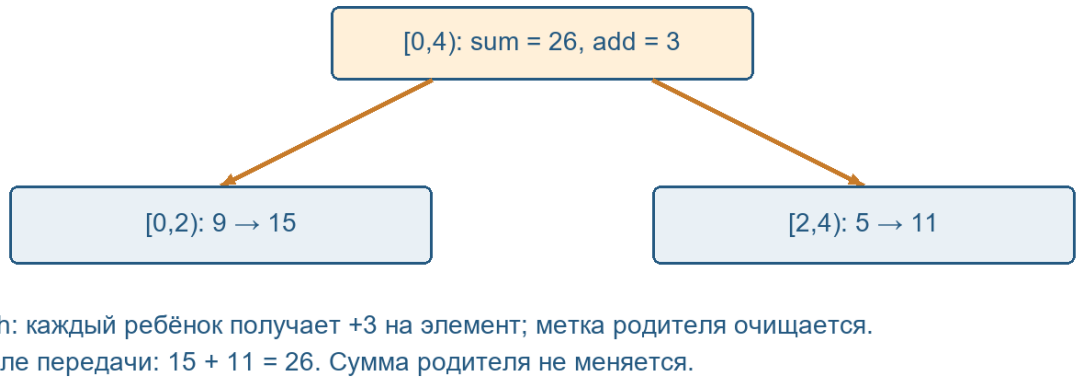


Рисунок 1: Полное покрытие меняет сумму и метку узла; перед частичным спуском метка передаётся детям.

Это соглашение устраняет типичную двойную прибавку: при полном покрытии возвращаем сохранённую сумму, не прибавляя метку ещё раз. Оно же объясняет, почему перед рекурсивным запросом нужно протолкнуть метку, даже если запрос не меняет логический массив.

(c) Aleksey Tolstikov

3.2. Сначала прибавление

Для прибавлений достаточно метки `pending`: сколько ещё нужно добавить каждому элементу детей. Чтобы применить d к узлу длины k, выполняем `sum += d*k` и `pending += d`. Чтобы протолкнуть накопленное p, применяем p к каждому ребёнку с его собственной длиной, затем обнуляем метку родителя.

Длины детей могут отличаться, если длина родительского интервала нечётна. У рекурсивного дерева на фактическом диапазоне $[l, r)$ середина равна $m = l + (r - l)/2$, поэтому длины — $m - l$ и $r - m$. Использование половины длины без проверки потеряет один элемент.

Пусть исходный массив $[2, 7, 1, 4, 3, 6, 5, 8]$. Добавим 3 к $[0, 4)$. Сумма левого ребёнка изменится с 14 до 26, его метка станет 3, общая сумма — 48. Листья этой половины пока не переписаны. Запрос всей половины сразу вернёт 26.

Для запроса $[2, 3)$ надо спуститься. Метка левого ребёнка передаётся его детям, а затем по нужному пути — листу. Значение позиции 2 становится доступно как 4. Другие поддеревья могут продолжать хранить собственные метки: проталкивание нужно по требованию, а не во всём дереве после каждого изменения.

Интервальная модификация не обещает более дешёвого вывода всего массива: вывод n значений сам требует $\Omega(n)$ времени. Выигрыш относится к операциям, результат которых можно описать компактной сводкой.

3.3. Присваивание ломает коммутативность

Добавления можно суммировать в любом порядке. С присваиванием это неверно. Для элемента $x = 2$ операции «присвоить 5, прибавить 3» дают 8. Операции «прибавить 3, присвоить 5» дают 5. Поэтому метка должна хранить именно функцию преобразования, а правило её обновления — порядок исполнения.

Есть удобное общее представление:

$$f(x) = ax + b, \qquad a \in \{0, 1\}.$$

Прибавление d — пара $(1, d)$; присваивание c — $(0, c)$; отсутствие изменения — $(1, 0)$. Если раньше отложена $g(x) = a_g x + b_g$, а теперь приходит $f(x) = a_f x + b_f$, детям позднее нужно применить сначала g, затем f:

$$f(g(x)) = (a_f a_g)x + (a_f b_g + b_f).$$

Порядок аргументов — часть контракта. Будем говорить «новая функция после старой». При полном покрытии обновим сумму как $S' = a_f S + b_f k$, а метку заменим композицией $f \circ g$.

Старая метка	Новая операция	Итоговая метка
Добавить p	Добавить d	Добавить $p + d$
Присвоить c	Добавить d	Присвоить $c + d$
Добавить p	Присвоить c	Присвоить c
Присвоить c	Присвоить z	Присвоить z

Нулевое присваивание — полноценная операция $(0, 0)$. Его нельзя путать с отсутствием метки $(1, 0)$. Использование одного числа с соглашением «ноль означает нет присваивания» сделало бы невозможным корректное присваивание нуля.

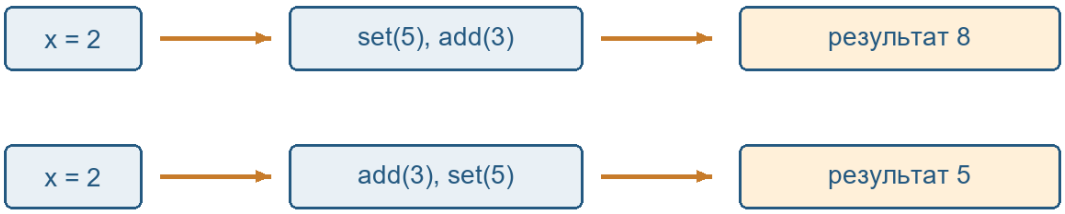


Рисунок 2: Композиция зависит от порядка: set(5), add(3) и add(3), set(5) дают разные результаты.

3.4. Реализация: один способ применять метку

Весь код, меняющий сумму и отложенную функцию узла, сосредоточен в `apply`. Такое устройство важно для корректности: обновление из пользовательской операции и обновление при проталкивании обязаны использовать одинаковое правило.

```
class LazySumTree {
    struct Node {
        long long sum = 0, mul = 1, add = 0;
    };
    int n;
    std::vector<Node> t;
    void build(int v, int l, int r, const std::vector<long long>& a) {
        if (r - l == 1) { t[v].sum = a[l]; return; }
        int m = l + (r - l) / 2;
        build(v * 2, l, m, a);
        build(v * 2 + 1, m, r, a);
        pull(v);
    }
    void pull(int v) { t[v].sum = t[v*2].sum + t[v*2+1].sum; }
    // Apply NEW transformation x -> a*x+b after the stored one.
    void apply(int v, int len, long long a, long long b) {
        t[v].sum = a * t[v].sum + b * len;
        t[v].mul = a * t[v].mul;
        t[v].add = a * t[v].add + b;
    }
    void push(int v, int l, int r) {
        int m = l + (r - l) / 2;
        apply(v*2, m-l, t[v].mul, t[v].add);
        apply(v*2+1, r-m, t[v].mul, t[v].add);
        t[v].mul = 1; t[v].add = 0;
    }
```

`push` вызывается только при частичном пересечении. Лист длины 1 при целочисленных границах не может пересекаться с запросом частично: он либо внутри, либо снаружи. Поэтому у него проталкивание не вызывается. Это свойство алгоритма, а не предположение о том, что обращение к несуществующим детям безвредно.

(c) Aleksey Tolstikov

Проталкивание сохраняет сумму родителя. Новые суммы детей после одного и того же аффинного преобразования в сумме равны прежней актуальной сумме родителя. После очистки родительской метки обычное равенство сумм вновь выполняется.

3.5. Изменение и запрос

Следующий листинг продолжает класс из пункта 3.4. Полный компилируемый файл с заголовками — `code/03-lazy-tree.hpp`.

```
    void update(int v, int l, int r, int ql, int qr,
                long long a, long long b) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) { apply(v, r-l, a, b); return; }
        push(v, l, r);
        int m = l + (r-l)/2;
        update(v*2, l, m, ql, qr, a, b);
        update(v*2+1, m, r, ql, qr, a, b);
        pull(v);
    }
    long long query(int v, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[v].sum;
        push(v, l, r);
        int m = l + (r-l)/2;
        return query(v*2, l, m, ql, qr)
             + query(v*2+1, m, r, ql, qr);
    }
    void bounds(int l, int r) const { assert(0 <= l && l <= r && r <= n); }
public:
    explicit LazySumTree(const std::vector<long long>& a)
        : n(int(a.size())), t(4 * (n > 0 ? n : 1)) {
        if (n) build(1, 0, n, a);
    }
    void add(int l, int r, long long x) {
        bounds(l,r);
        if (l < r) update(1,0,n,l,r,1,x);
    }
    void set(int l, int r, long long x) {
        bounds(l,r);
        if (l < r) update(1,0,n,l,r,0,x);
    }
    long long sum(int l, int r) {
        bounds(l,r);
        return l == r ? 0 : query(1,0,n,l,r);
    }
};
```

При полном покрытии обновление заканчивается вызовом `apply`. При частичном сначала делаем детей актуальными относительно родителя, затем рекурсивно изменяем нужные части и пересчитываем сумму. Пересчёт **до** передачи метки затёр бы ещё не отражённые в детях изменения.

Запрос также проталкивает метки, поэтому в реализации он не `const`. Логический массив не меняется; меняется только распределение отложенной работы внутри представления. Публичный интерфейс не обещает неизменности внутренней памяти при чтении.

Для пустого массива конструктор не строит дерево. Пустые операции завершаются в публичных методах и не запускают рекурсию по диапазону нулевой длины. Это предотвращает бесконечное деление пустого интервала.

Доказательство обновления. Вне запроса состояние не меняется. При полном покрытии формула суммы точна для всех элементов диапазона, а композиция сохраняет обязанность перед детьми. При частичном покрытии `push` восстанавливает нужное соответствие детей текущему состоянию; индуктивное предположение применяется к обоим рекурсивным вызовам; `pull` объединяет правильные суммы. Корень поэтому всегда представляет весь логический массив.

Как и в предыдущей главе, на каждом уровне не более двух частично пересечённых узлов. Полностью покрытые поддеревья не обходятся, `apply` и `push` постоянны. Значит, каждая из трёх операций имеет $O(\log n)$ времени в худшем случае, построение и память — $O(n)$.

3.6. Подробная трасса смешанных операций

Начнём с $[2, 7, 1, 4]$. Таблица описывает **логическое** содержимое; листья дерева не обязаны сразу принимать эти значения.

Операция	Логический массив	Сумма всего
Исходное состояние	2, 7, 1, 4	14
`set(0,4,5)`	5, 5, 5, 5	20
`add(1,3,2)`	5, 7, 7, 5	24
`set(2,4,0)`	5, 7, 0, 0	12
`add(0,4,-3)`	2, 4, −3, −3	0

После первого шага только корень обязательно имеет сумму 20 и метку $(0, 5)$. Второй шаг частично пересекает корень, поэтому присваивание передаётся обеим половинам. Дальше аналогичное проталкивание происходит в половинах, чтобы отдельно изменить позиции 1 и 2.

Третий шаг целиком покрывает правую половину. Новая метка $(0, 0)$ уничтожает накопленную историю изменений этой половины: всем её элементам назначено нулевое значение независимо от прошлого. Последнее прибавление можно оставить у корня; запрос `sum(1,3)` протолкнёт его по нужным путям и вернёт $4 + (-3) = 1$.

Эта короткая трасса содержит отрицательное прибавление, присваивание нуля, полное и частичное покрытия. Она полезнее теста только из положительных добавлений: в нём неверная композиция присваиваний вообще не проявилась бы.

3.7. Пример курса: чётные и нечётные суммы

В старом конспекте `segment_tree.tex`, раздел 2.6.2, дан интерфейс: присвоить значение элементу, прибавить 1 на диапазоне и запросить отдельно суммы чётных и нечётных элементов. Это же содержательное усложнение встречается в iRunner 3764. Здесь одной общей суммы недостаточно.

Храним четыре величины: число чётных c_e, число нечётных c_o, сумму чётных s_e и сумму нечётных s_o. При прибавлении единицы каждый чётный становится нечётным и наоборот:

$$s'_e = s_o + c_o, \quad s'_o = s_e + c_e, \quad c'_e = c_o, \quad c'_o = c_e.$$

Все правые части должны использовать **старые** значения. Если сначала перезаписать s_e, а затем использовать его для вычисления s_o, получится двойное применение изменения.

Например, на $[2, 3, 4]$ начальные сведения: $c_e = 2$, $s_e = 6$, $c_o = 1$, $s_o = 3$. После прибавления 1 массив равен $[3, 4, 5]$; новые чётная и нечётная суммы равны 4 и 8. Формулы дают $3 + 1 = 4$ и $6 + 2 = 8$.

Для произвольного прибавления d сначала увеличиваются суммы на d умножить на соответствующие количества. Если d нечётно, две категории меняются местами. В C++ для проверки нечётности отрицательного числа используйте `d % 2 != 0`, а не `d % 2 == 1`. Композиция таких прибавлений по-прежнему складывает d.

Точечное присваивание спускается до листа, предварительно проталкивая метки предков. Лист получает одну ненулевую категорию по чётности нового значения. После этого сводки предков объединяются покомпонентным сложением. Реализацию этого расширения оставляем самостоятельной: она проверяет понимание инварианта, а не копирование готового шаблона.

Другой пример старого конспекта — перекраска полоски с запросом цвета точки. Если нужны только присваивания диапазона и чтение одной позиции, хранить сумму не требуется. Более слабый интерфейс допускает более простую сводку. Не добавляйте поля только потому, что они были в предыдущем решении.

3.8. Геометрия, ограничения и проверка

Интервальные структуры естественно появляются в методе сканирующей прямой. Например, при подсчёте площади объединения прямоугольников вертикальные границы создают события добавления и удаления интервалов по y. Между соседними событиями площадь увеличивается на длину покрытого объединения, умноженную на расстояние по x.

Однако длина объединения не является обычной суммой счётчиков покрытий. Нужны число покрывающих весь узел интервалов и покрытая длина. Если счётчик положителен, покрыт весь геометрический интервал; иначе длина складывается из детей. После сжатия координат длина листа равна разности соседних координат, а не единице. Это отдельная сводка с отдельным доказательством, а не автоматическое применение приведённого класса.

Для основного класса эталон — обычный вектор. Выполняйте каждую модификацию поэлементно, после неё сравнивайте диапазонные суммы. Обязательно включите последовательности `set→add`, `add→set`, `set→set`, присваивание нуля, изменение всего массива перед изменением одного элемента, вложенные и непересекающиеся диапазоны.

Происхождение. Основной механизм и примеры перекраски и чётности опираются на локальный конспект дерева отрезков. Числовая трасса и единая аффинная запись меток — редакторская разработка для этой главы. В записи 20.09.2022, 45:00–48:00, рассмотрен другой приём — глобальное присваивание через версии ячеек; его разбираем отдельно ниже.

3.9. Разбор: что происходит при частичном запросе

Начнём с $[2, 7, 1, 4]$ и выполним присваивание 5 на всём массиве. В корне теперь сумма 20 и метка $(0, 5)$. Значения детей могут по-прежнему содержать старые суммы 9 и 5. Это допустимо: корень уже отвечает правильно за весь диапазон, а метка объясняет, какое преобразование ещё не отражено ниже.

Запрос $[1, 3)$ не совпадает с корнем. Сначала `push` применяет присваивание к обоим детям. Их суммы становятся по 10, их метки — $(0, 5)$, а метка корня очищается до $(1, 0)$. Сумма корня остаётся 20. Затем запрос спускается в обе половины и аналогично передаёт присваивание листьям. Листья 1 и 2 возвращают по 5; результат равен 10.

Теперь выполним прибавление 2 на $[1, 3)$. Листья 1 и 2 становятся равны 7. При возврате пересчитываются суммы половин: обе равны 12, сумма корня — 24. Если бы мы пересчитали корень сразу после исходного полного присваивания, до передачи метки, получили бы старую сумму 14. Ошибка возникает не в сложении, а в нарушении условия, при котором сведения детей уже актуальны.

Момент	Сумма корня	Суммы детей	Метка корня
Исходный массив	14	9, 5	$(1, 0)$
Полное присваивание 5	20	9, 5	$(0, 5)$
Передача к детям	20	10, 10	$(1, 0)$
Частичное прибавление 2	24	12, 12	$(1, 0)$

Эта таблица — редакторская трасса реализации. Её полезно воспроизвести отладочным выводом узлов, а затем убрать вывод перед измерением времени. Не сравнивайте каждый внутренний узел напрямую с массивом без учёта меток предков: ещё не переданное преобразование может законно делать его локальную запись устаревшей относительно всего массива.

Для независимой проверки сравнивайте публичные ответы с простым вектором. Чтобы дополнительно проверить внутреннее представление, можно сначала передать все метки до листьев, затем проверить их значения и восстановленные суммы. Такой полный обход стоит $O(n)$ и относится к диагностике, а не к обычному запросу структуры.

3.10. Углубление: какие преобразования допускает сводка

Наличие функции над отдельным элементом ещё не означает, что её можно быстро применить к узлу. Требуется вычислять новую сводку по старой сводке и метке, не читая все элементы диапазона. Для суммы и прибавления это возможно, если известна длина: $S' = S + d \cdot len$. Без длины суммы одинаковых значений не отличают массив $[4]$ от $[2, 2]$, но после прибавления единицы их суммы различаются.

Для присваивания достаточно той же длины: $S' = x \cdot len$. Для аффинного преобразования $x \mapsto ax + b$ получаем $S' = aS + b \cdot len$. В нашей реализации используются только $a = 0$ и $a = 1$. Разрешение произвольного умножения потребует пересмотра границ чисел: многократная композиция быстро увеличивает коэффициенты. В модульной арифметике можно хранить коэффициенты по модулю, но это уже другой числовой контракт.

У корректного действия меток есть три свойства. Единичная метка оставляет сводку неизменной. Применение старой, затем новой метки совпадает с применением их композиции. Наконец, обработка объединённого диапазона совпадает с объединением обработанных половин с их собственными длинами. Последнее свойство связывает алгебру меток с алгеброй `merge` и объясняет, почему `push` сохраняет сумму родителя.

Контрпример особенно полезен для новых операций. Пусть требуется заменить каждое значение на $\min(x, 3)$, а узел хранит только сумму и длину. Массивы $[0, 6]$ и $[3, 3]$ имеют одинаковую длину 2 и сумму 6. После преобразования их суммы равны 3 и 6. Значит, по прежней сводке невозможно восстановить ответ: никакая формула от этих двух чисел не исправит потерю информации.

Это не доказательство невозможности эффективной структуры вообще. Можно хранить дополнительные сведения и применять преобразование только при специальных условиях; такие конструкции требуют отдельного анализа. Но обычный шаблон lazy propagation не превращает произвольное поэлементное изменение в логарифмическую операцию.

Похожая ловушка — одновременно поддерживать прибавление на диапазоне и максимальную сумму подотрезка из второй главы. Кандидат длины k изменяет сумму на $d \cdot k$, поэтому после прибавления

может победить другой подотрезок. Четырёх полей `sum`, `pref`, `suff`, `best` недостаточно для механического обновления прежнего оптимума. Сначала докажите замкнутость сводки относительно преобразования; только затем пишите `apply`.

Самостоятельная проверка модели. Для суммы, минимума и пары «минимум, число вхождений» определите, какие из операций `add`, `set`, умножение на положительное число и умножение на ноль допускают обновление за $O(1)$. Для каждого разрешённого случая выпишите формулу. Обратите внимание: умножение на ноль делает все элементы минимальными, поэтому прежней кратности минимума недостаточно без знания длины.

3.11. Пример из видео: глобальное присваивание через версии

В расшифрованном фрагменте лекции 20.09.2022, примерно 46:07–47:58, предлагаются два массива: значения и версии ячеек. Структура хранит также текущую версию и общее значение по умолчанию. Это особенно простой пример отложенного изменения: операция затрагивает весь массив, а чтение обращается только к одной позиции.

Обозначим текущую версию $epoch$, значение по умолчанию $default$, локальные записи $value[i]$ и $stamp[i]$. После `assign_all(x)` увеличиваем $epoch$ и устанавливаем $default = x$. После `set(i,x)` записываем $value[i] = x$ и $stamp[i] = epoch$. При `get(i)` возвращаем локальное значение, если версии совпадают, иначе — общее значение по умолчанию. Последнее правило следует из инварианта; конец выбранного звукового фрагмента обрывается во время его объяснения.

Числовой пример ниже редакторский. Пусть размер равен 3, начальное значение 0. После `set(1,7)` логический массив равен $[0, 7, 0]$. Затем `assign_all(5)` делает его равным $[5, 5, 5]$, хотя число 7 физически остаётся в старой ячейке. Версия этой ячейки больше не актуальна, поэтому чтение не возвращает 7. После `set(2,9)` получаем $[5, 5, 9]$; только позиция 2 имеет локальную запись текущей версии.

Инвариант разделяет два случая: ячейка либо записана после последнего глобального присваивания, либо наследует его значение. Присваивание всему массиву меняет границу актуальности сразу для всех ячеек, а точечная запись переводит одну ячейку в первый случай. После начального выделения и инициализации $O(n)$ все три операции постоянны по времени, память линейна.

Счётчик версий не должен переполняться: иначе старый `stamp` может снова совпасть с текущим. Контракт ограничивает число глобальных присваиваний. Полный сброс штампов при исчерпании счётчика возможен, но стоит $O(n)$ и меняет гарантию отдельной операции.

Приём обслуживает глобальное присваивание и точечное чтение. Произвольные перекрывающиеся диапазоны требуют отдельных отложенных состояний в разных частях — такую иерархию создаёт дерево. Общая идея сохраняется: логическое изменение не обязано немедленно переписывать все ячейки.

3.12. Практика к главе

В расширенной подборке приложения A: №4488 «Билеты» (основная); №3764 «Сумма на отрезке» (основная); №4492 «Картина» (углубление). Полные условия доступны по ссылкам в приложении. Выберите основные задачи, затем переходите к углублению; решать весь список за одну неделю не требуется.

3.13. Вопросы для самопроверки

1. Что уже учтено в сумме узла, а что ещё не учтено в детях?
2. Почему запрос при полном покрытии не должен прибавлять метку ещё раз?
3. Когда равенство суммы родителя и сумм детей временно неверно?
4. Получите формулу композиции новой функции после старой.
5. Чем отличаются пары $(0, 0)$ и $(1, 0)$?
6. Почему частичный запрос может менять внутреннее состояние дерева?
7. В массиве [1,2,3,4] выполните add на [0,3) со значением 1, затем set на [1,4) со значением 0. Какова сумма на [0,4), и почему обратный порядок операций даёт другой ответ?
8. Выведите формулы обновления чётных и нечётных сумм.
9. Почему высоты дерева недостаточно для оценки, если полное покрытие не останавливает спуск?
10. Какие дополнительные сведения нужны для длины объединения геометрических интервалов?

(c) Aleksey Tolstikov